In [40]:
```
import pandas as pd
from mlba import load_data
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from pandas.api.types import CategoricalDtype
import matplotlib.pyplot as plt
import matplotlib
%matplotlib inline
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, roc_cu
from sklearn.tree import plot_tree
from sklearn import tree
```

In [41]:
```
#VB67
# Loading dataset
df = load_data('eBayAuctions.csv')

#knowing the shape
print("Shape:", df.shape)
df.head()
```

```
Shape: (1972, 8)
```

Out[41]:

	Category	currency	sellerRating	Duration	endDay	ClosePrice	OpenPrice
0	Music/Movie/Game	US	3249	5	Mon	0.01	0.01
1	Music/Movie/Game	US	3249	5	Mon	0.01	0.01
2	Music/Movie/Game	US	3249	5	Mon	0.01	0.01
3	Music/Movie/Game	US	3249	5	Mon	0.01	0.01
4	Music/Movie/Game	US	3249	5	Mon	0.01	0.01

In [42]:
```
#VB67
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1972 entries, 0 to 1971
Data columns (total 8 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Category      1972 non-null   object
 1   currency      1972 non-null   object
 2   sellerRating  1972 non-null   int64
 3   Duration      1972 non-null   int64
 4   endDay        1972 non-null   object
 5   ClosePrice    1972 non-null   float64
 6   OpenPrice     1972 non-null   float64
 7   Competitive?  1972 non-null   int64
dtypes: float64(2), int64(3), object(3)
memory usage: 123.4+ KB
```

In [43]:
```
#VB67
#displaying competitive column
y = df['Competitive?']
#displaying last 5 rows
y.tail()
```

Out[43]:
```
1967    0
1968    1
1969    0
1970    1
1971    0
Name: Competitive?, dtype: int64
```

Data Preprocessing

In [44]:
```
#VB67
# Converting Duration to categorical variable
df['Duration'] = df['Duration'].astype('category')

# Checking that 'Competitive' exists
print("Columns:", df.columns)

# Separating features and target
X = df.drop(columns=['Competitive?'])
y = df['Competitive?']

# Train-test split (60,40)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)

print("Training set:", X_train.shape)
print("Test set:", X_test.shape)
```

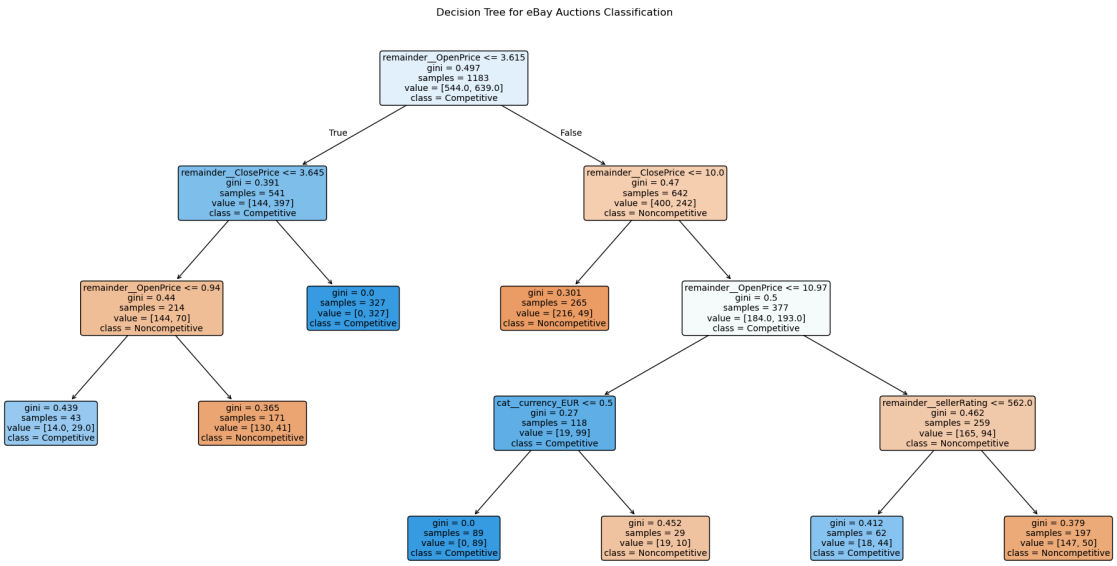
```
Columns: Index(['Category', 'currency', 'sellerRating', 'Duration', 'endDa
y',
       'ClosePrice', 'OpenPrice', 'Competitive?'],
      dtype='object')
Training set: (1183, 7)
Test set: (789, 7)
```

a.

In [38]:
```
#VB67
RANDOM_STATE = 42

# Identifying categorical and numeric columns
cat_cols = [c for c in X_train.columns if isinstance(X_train[c].dtype, Categ
num_cols = [c for c in X_train.columns if c not in cat_cols]

# Preprocessing: One-hot encode categorical columns
preprocessor = ColumnTransformer(
```

```
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
    ],
    remainder='passthrough'
)

# Defining classifier
clf = DecisionTreeClassifier(
    min_samples_split=50,
    max_depth=7,
    min_impurity_decrease=0.01,
    random_state=RANDOM_STATE
)

# Pipeline
pipe = Pipeline([
    ('pre', preprocessor),
    ('clf', clf)
])

# Fitting model
pipe.fit(X_train, y_train)

# Printing rules
feature_names = pipe.named_steps['pre'].get_feature_names_out()
rules = export_text(pipe.named_steps['clf'], feature_names=list(feature_name
print(rules)

Plot the decision tree ===
model = pipe.named_steps['clf']
feature_names = pipe.named_steps['pre'].get_feature_names_out()

plt.figure(figsize=(25, 12))
plot_tree(
    model,
    feature_names=feature_names,
    class_names=['Noncompetitive', 'Competitive'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("Decision Tree for eBay Auctions Classification")
#display tree
plt.show()
```

```
|--- remainder__OpenPrice <= 3.62
|   |--- remainder__ClosePrice <= 3.64
|   |   |--- remainder__OpenPrice <= 0.94
|   |   |   |--- class: 1
|   |   |--- remainder__OpenPrice >  0.94
|   |   |   |--- class: 0
|   |--- remainder__ClosePrice >  3.64
|   |   |--- class: 1
|--- remainder__OpenPrice >  3.62
|   |--- remainder__ClosePrice <= 10.00
|   |   |--- class: 0
|   |--- remainder__ClosePrice >  10.00
|   |   |--- remainder__OpenPrice <= 10.97
|   |   |   |--- cat__currency_EUR <= 0.50
|   |   |   |   |--- class: 1
|   |   |   |--- cat__currency_EUR >  0.50
|   |   |   |   |--- class: 0
|   |   |--- remainder__OpenPrice >  10.97
|   |   |   |--- remainder__sellerRating <= 562.00
|   |   |   |   |--- class: 1
|   |   |   |--- remainder__sellerRating >  562.00
|   |   |   |   |--- class: 0
```



b.

- Yes, because it uses only features known before the auction ends:
  - Category, currency, sellerRating, Duration, OpenPrice, endDay
- The model can help sellers estimate if their auction is likely to be competitive.
- Caution: Some splits may be overly specific (rare categories), so pruning or simplifying might improve generalization.

c.

- Interesting: Rules highlighting the effect of:

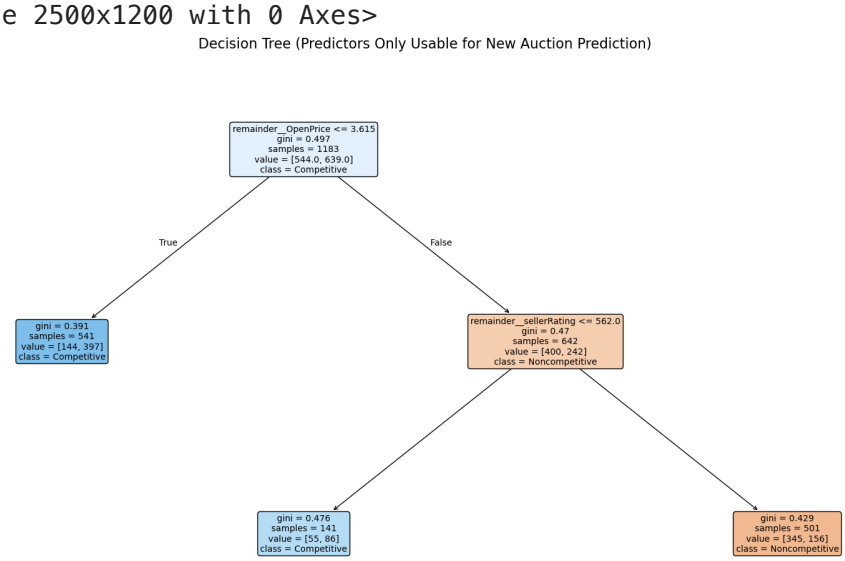
    - Seller rating - higher rating increases competitiveness
    - Opening price - lower starting bids attract more bids
    - Duration - medium-length auctions (5–7 days) tend to be more competitive
- Uninteresting: Rules splitting on rare categories or end days with few auctions
- Insights: Sellers can adjust auction settings to improve competitiveness

d.

In [45]:
```
#VB67
# Removing features not known beforehand
X2 = X_train.copy()
X2_test = X_test.copy()
for col in ['ClosePrice', 'ClosePriceCurrency']:
    if col in X2.columns:
        X2.drop(columns=[col], inplace=True)
        X2_test.drop(columns=[col], inplace=True)

# Identifying categorical columns
cat_cols2 = [c for c in X2.columns if isinstance(X2[c].dtype, CategoricalDty

# Preprocessing
preprocessor2 = ColumnTransformer(
    transformers=[('cat', OneHotEncoder(handle_unknown='ignore', sparse_outp
    remainder='passthrough'
)

# Tree classifier
clf2 = DecisionTreeClassifier(
    min_samples_split=50,
    max_depth=7,
    min_impurity_decrease=0.01,
    random_state=RANDOM_STATE
)

pipe2 = Pipeline([
    ('pre', preprocessor2),
    ('clf', clf2)
])

# Fitting model
pipe2.fit(X2, y_train)

# Printing minimal rule set
feature_names2 = pipe2.named_steps['pre'].get_feature_names_out()
rules2 = export_text(pipe2.named_steps['clf'], feature_names=list(feature_na
print(rules2)

# Extracting the fitted tree model and feature names
model2 = pipe2.named_steps['clf']
feature_names2 = pipe2.named_steps['pre'].get_feature_names_out()

# Plotting the tree
plt.figure(figsize=(25, 12))
```

```
tree.plot_tree(
    model2,
    feature_names=feature_names2,
    class_names=['Noncompetitive', 'Competitive'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("Decision Tree (Predictors Only Usable for New Auction Prediction)
#Display plot
plt.show()
```

```
|--- remainder__OpenPrice <= 3.62
|   |--- class: 1
|--- remainder__OpenPrice >  3.62
|   |--- remainder__sellerRating <= 562.00
|   |   |--- class: 1
|   |--- remainder__sellerRating >  562.00
|   |   |--- class: 0

<Figure size 2500x1200 with 0 Axes>
```

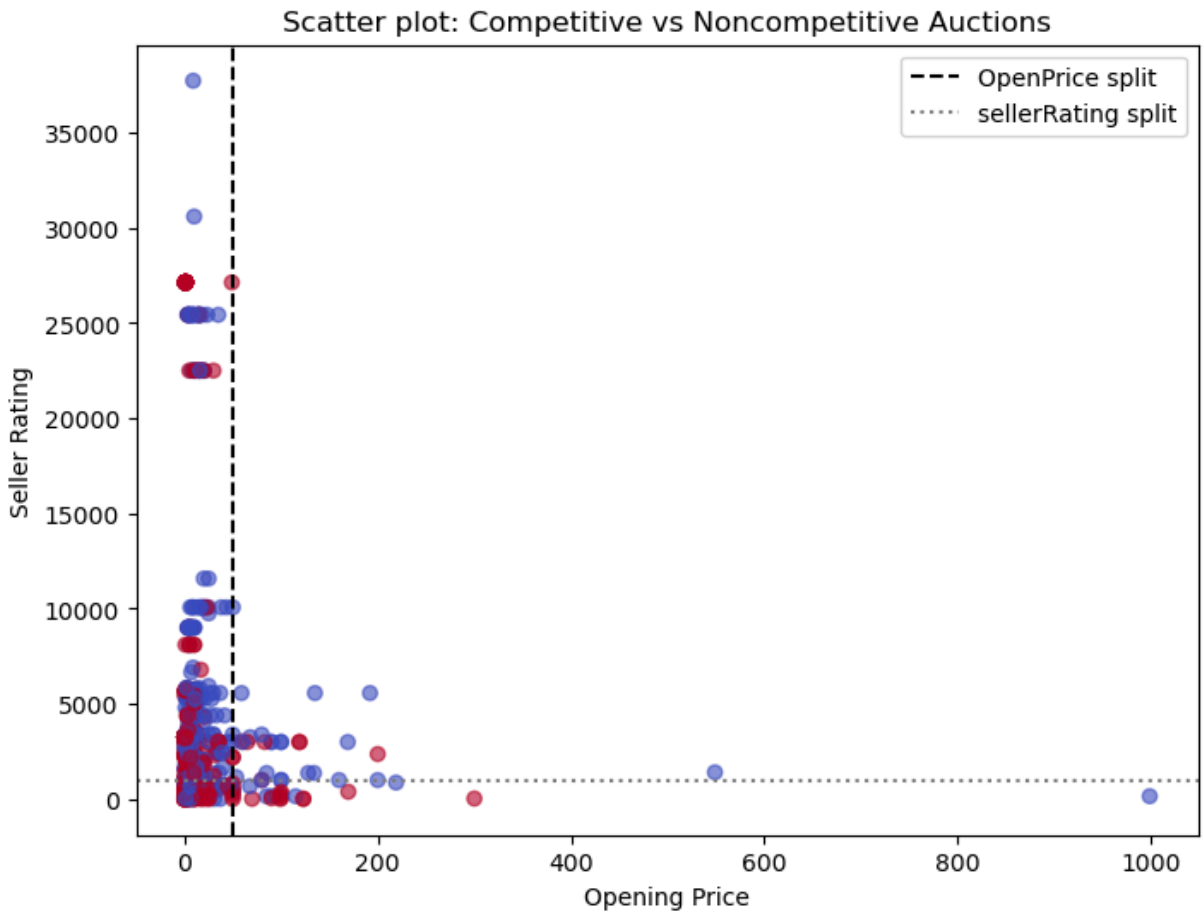


e.

In [46]:
```
#VB67
# numeric predictors: OpenPrice and sellerRating
plt.figure(figsize=(8,6))
plt.scatter(X2['OpenPrice'], X2['sellerRating'], c=y_train, cmap='coolwarm',
plt.xlabel('Opening Price')
plt.ylabel('Seller Rating')
plt.title('Scatter plot: Competitive vs Noncompetitive Auctions')

# split lines (from tree rules)
plt.axvline(x=50, color='black', linestyle='--', label='OpenPrice split')
plt.axhline(y=1000, color='gray', linestyle=':', label='sellerRating split')
plt.legend()
```

Out[46]: <matplotlib.legend.Legend at 0x15500e0d0>

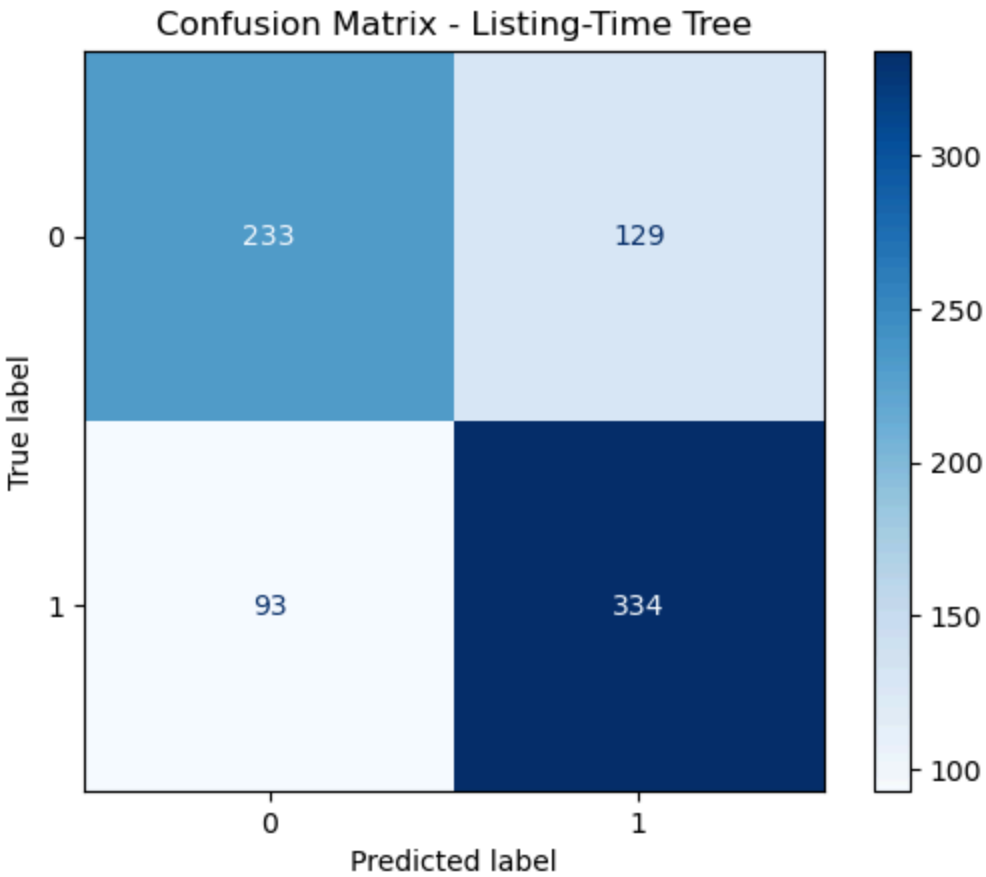
In [47]:
```
#VB67
#Display plot
plt.show()
```

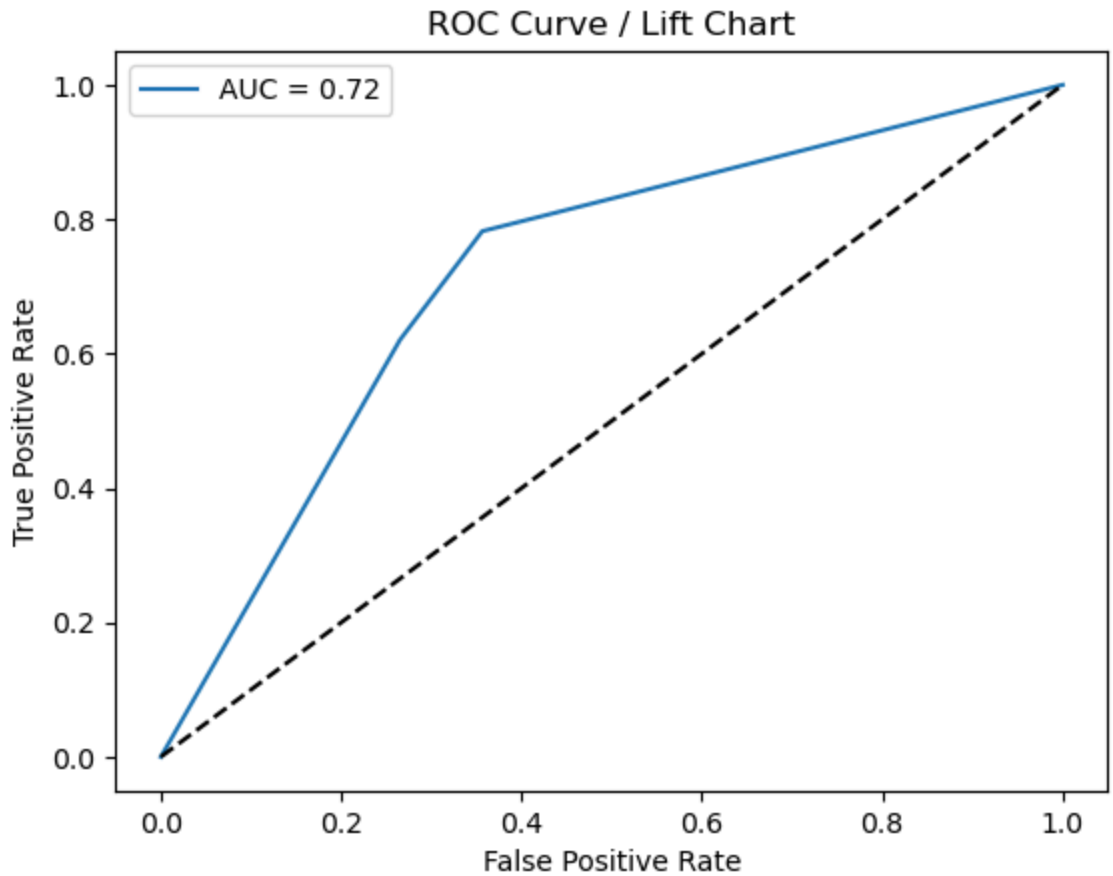


f.

In [31]:
```
#VB67
# Predictions
y_pred = pipe2.predict(X2_test)
y_prob = pipe2.predict_proba(X2_test)[:, 1]

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm)
disp.plot(cmap='Blues')
plt.title("Confusion Matrix - Listing-Time Tree")
plt.show()

# ROC Curve / Lift chart
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0,1], [0,1], 'k--')
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
plt.title("ROC Curve / Lift Chart")
plt.legend()
# Display plot
plt.show()
```





g.

Low opening prices attract more bids. Higher seller ratings increase auction competitiveness. Medium-length durations (5–7 days) are optimal. Sellers should:

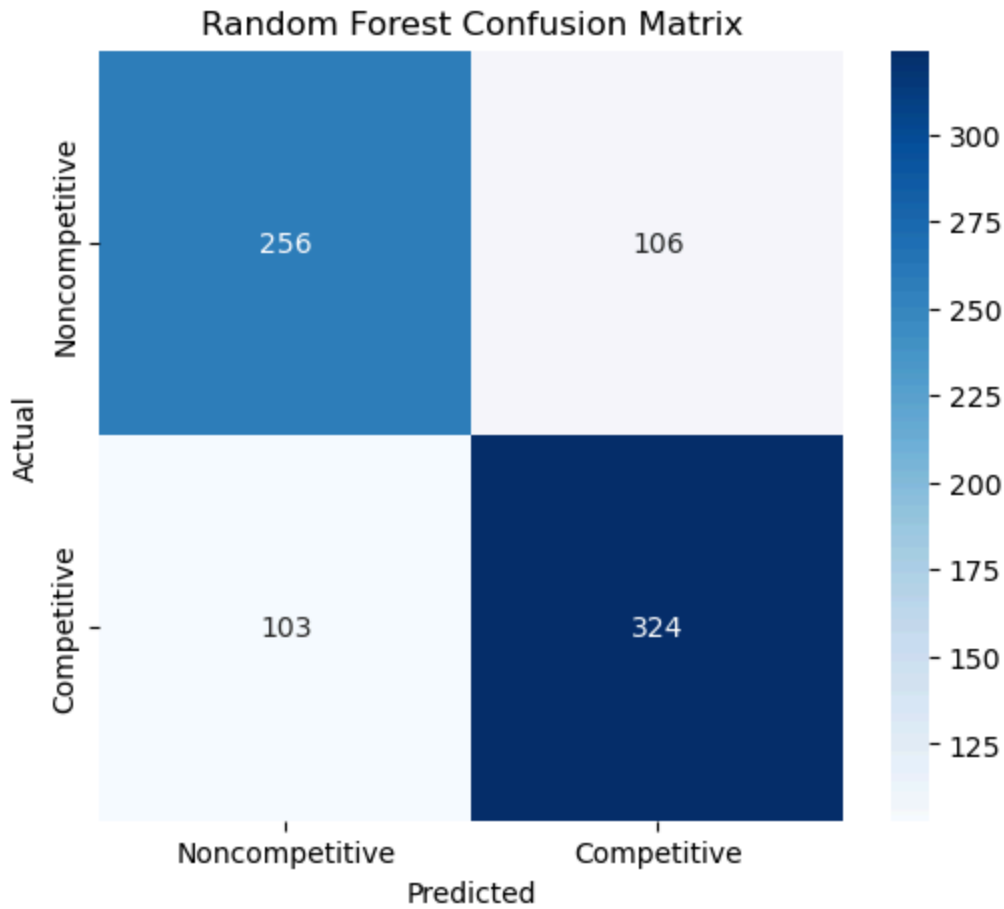
- Set a lower starting price.
- Maintain a high seller rating.
- Choose moderate auction durations.
- End auctions on popular traffic days.

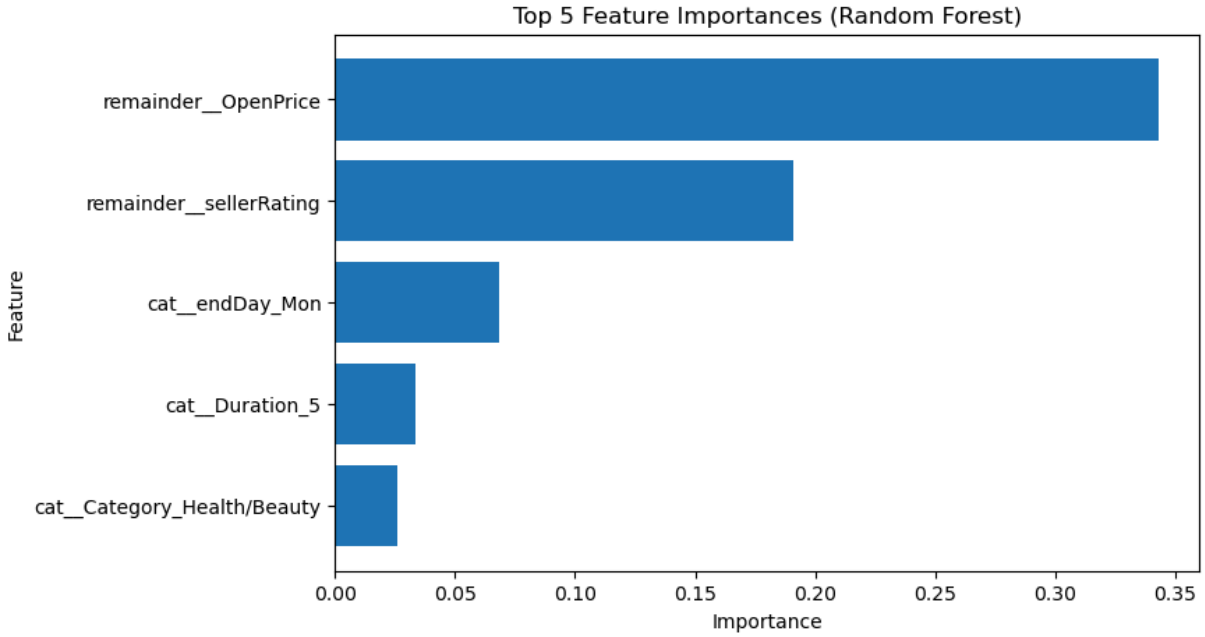
Add-on question

In [48]:
```
#VB67
# Random Forest classifier using predictors from 9.1(d) ===
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt

# Defining Random Forest model
rf_clf = RandomForestClassifier(
    n_estimators=200,
    max_depth=7,
```

```
    min_samples_split=50,
    random_state=RANDOM_STATE
)

# Pipeline: same preprocessing as before
pipe_rf = Pipeline([
    ('pre', preprocessor2),   # use preprocessor from pipe2
    ('rf', rf_clf)
])

# Fitting model on training data
pipe_rf.fit(X2, y_train)

# Predictions on holdout dataset
y_pred_rf = pipe_rf.predict(X2_test)

# Confusion matrix and accuracy
cm = confusion_matrix(y_test, y_pred_rf)
acc = accuracy_score(y_test, y_pred_rf)

print("=== Random Forest Results ===")
print("Confusion Matrix:\n", cm)
print(f"Accuracy on holdout data: {acc:.3f}")

# Confusion Matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Noncompetitive', 'Competitive'],
            yticklabels=['Noncompetitive', 'Competitive'])
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Feature Importance Plot
# Extract feature importances
importances = pipe_rf.named_steps['rf'].feature_importances_
feature_names_rf = pipe_rf.named_steps['pre'].get_feature_names_out()

# Top 5 important features
indices = np.argsort(importances)[-5:]
plt.figure(figsize=(8, 5))
plt.barh(range(len(indices)), importances[indices], align='center')
plt.yticks(range(len(indices)), [feature_names_rf[i] for i in indices])
plt.title("Top 5 Feature Importances (Random Forest)")
plt.xlabel("Importance")
plt.ylabel("Feature")
#display plot
plt.show()
```

```
=== Random Forest Results ===
Confusion Matrix:
 [[256 106]
 [103 324]]
Accuracy on holdout data: 0.735
```


Random Forest Confusion Matrix
Noncompetitive
Competitive
Actual
256
106
103
324
Noncompetitive
Competitive
Predicted
300
275
250
225
200
175
150
125


Top 5 Feature Importances (Random Forest)
remainder__OpenPrice
remainder__sellerRating
cat__endDay_Mon
cat__Duration_5
cat__Category_Health/Beauty
Feature
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
Importance

In [ ]: